

Examen Parcial — Unidad 1

Versión B · Caso: BiblioAndes, préstamos de biblioteca

Dato	Detalle	Dato	Detalle
Asignatura	Desarrollo de Aplicaciones Móviles	Ciclo	VI · Semestre 2026-2
Unidad	1. Fundamentos de KMP y UI con Compose Multiplatform	Sesión	6 (evaluación de unidad)
Modalidad	Producto en equipo (2) + defensa individual	Duración	4 horas continuas (2 teóricas + 2 prácticas)
Nivel	Intermedio · Grupo B	Puntaje	20 puntos

1. Indicaciones generales

Este examen evalúa el Producto de la Unidad 1: «Aplicación KMP ejecutándose en Android e iOS con navegación completa, theming Material 3 y arquitectura Clean + MVVM con datos simulados». Es equivalente en exigencia y puntaje a la versión A, y se diferencia en el caso de negocio y en el nivel de andamiaje que se te entrega.

Parte	Momento	Qué se evalúa	Puntaje
Parte I. Producto del caso	Sesiones 1 a 5; se demuestra en los primeros 45 minutos.	Que la aplicación cumpla los requerimientos funcionales y técnicos de este caso.	12 pts
Parte II. Solicitud de cambio	120 minutos en aula; cada estudiante recibe una solicitud distinta.	Capacidad individual de modificar la aplicación respetando la arquitectura.	4 pts
Parte III. Defensa técnica	45 minutos, unos 15 por estudiante, al cierre de la sesión.	Comprensión de las decisiones de diseño y de la arquitectura implementada.	2 pts
Criterio transversal. Trabajo colaborativo con Git	Toda la unidad; se verifica en el historial del repositorio.	Que el desarrollo sea compartido y trazable: una rama y commits propios por integrante.	2 pts

1.1 Distribución del tiempo

El examen se desarrolla en una única jornada de cuatro horas continuas. Los tiempos son estrictos: al cerrarse cada bloque se evalúa lo que exista en el repositorio en ese momento.

Bloque	Duración	Actividad
1	15 min	Indicaciones, verificación de que cada equipo tiene su repositorio actualizado y su entorno operativo.
2	45 min	Parte I. Cada equipo demuestra la aplicación ejecutándose en Android e iOS y recorre los siete requerimientos funcionales.
3	120 min	Parte II. Cada estudiante implementa su solicitud de cambio en su propia rama, con un mínimo de tres commits.
4	45 min	Parte III. Defensa técnica individual, unos quince minutos por estudiante.
5	15 min	Cierre: subida de ramas, etiquetado del commit evaluado y entrega de evidencias.

1.2 Condiciones

- La aplicación debe ejecutarse en un emulador o dispositivo Android y en un simulador o dispositivo iOS. No se califica una aplicación que solo corre en una plataforma.
- Se permite consultar la documentación oficial de Kotlin, Compose Multiplatform y Material 3, así como el propio repositorio del equipo.
- No se permite copiar código de otro equipo ni recibir ayuda externa durante la Parte II.
- El uso de asistentes de inteligencia artificial está permitido solo si el estudiante explica y justifica cada línea durante la defensa. Si no puede explicar su propio código, esa parte se califica con cero.
- El desarrollo debe ser colaborativo y trazable en Git: cada integrante trabaja en su propia rama y fusiona su aporte al repositorio común. Las condiciones completas están en la sección 5.
- Durante el examen, cada estudiante debe registrar como mínimo tres commits propios en su rama, distribuidos a lo largo del bloque de trabajo. Un solo commit al final no acredita el proceso.
- Un integrante sin commits propios en el repositorio no puede aprobar el examen, aunque el producto del equipo funcione.

Prohibición explícita

Esta versión no puede consumir servicios web ni bases de datos. Está prohibido usar Ktor, SQLDelight, Room, Retrofit o cualquier librería de red o persistencia: todos los datos provienen de una fuente simulada en memoria. Ese es precisamente el punto del caso.

2. Presentación del caso

2.1 La organización

BiblioAndes es la biblioteca de un instituto tecnológico con tres sedes y alrededor de mil ochocientos estudiantes. El catálogo se consulta en una computadora ubicada en el mostrador y los préstamos se anotan en un cuaderno: el estudiante debe acercarse físicamente para saber si un libro está disponible y, con frecuencia, descubre que el único ejemplar ya fue prestado.

El personal tampoco tiene una manera sencilla de avisar cuándo vence un préstamo, así que las devoluciones tardías son la norma y los libros más solicitados permanecen fuera de circulación durante semanas.

2.2 La restricción del proyecto

La dirección aprobó construir una aplicación móvil para que el estudiante consulte el catálogo, solicite préstamos y controle sus fechas de devolución. El servicio web que expondrá el catálogo, sin embargo, todavía no existe: se desarrollará durante la segunda unidad del curso.

Tu equipo debe entregar la aplicación completa funcionando con datos simulados en memoria, organizada de modo que, cuando el servicio esté disponible, baste con reemplazar la implementación del repositorio por una que consuma la API, sin tocar la interfaz de usuario ni los casos de uso.

El criterio central de este examen

No se evalúa únicamente que la aplicación funcione. Se evalúa que esté organizada de modo que el cambio de datos simulados a datos reales sea una sustitución localizada y no una reescritura. Ese es el valor que la arquitectura Clean + MVVM aporta al cliente.

2.3 Nivel y andamiaje entregado

Esta versión está calibrada para un nivel intermedio. Eso no reduce el puntaje ni la exigencia técnica, pero sí te entrega resuelto el trabajo de análisis previo: el modelo de dominio ya está definido en la sección 4.2 y los datos semilla en el anexo. Tu tarea es implementarlo correctamente y organizarlo en capas, no diseñarlo desde cero.

- El modelo de dominio se te entrega escrito; debes implementarlo tal cual, respetando tipos y anulabilidad.
- Puedes reutilizar el esqueleto de navegación y el tema construidos en las sesiones 4 y 5, siempre que sean de tu propio repositorio.
- Las cuatro reglas de negocio están enunciadas de forma operativa: cada una se traduce en una condición verificable.
- A cambio, se exige rigor en la ubicación del código: una regla escrita dentro de un composable se califica como no lograda, aunque funcione.

3. Requerimientos funcionales

La aplicación debe implementar los siete requerimientos siguientes. Cada uno es verificable por observación directa durante la evaluación.

Código	Requerimiento	Comportamiento esperado
RF-01	Pantalla de inicio	Saludo con el nombre del estudiante, tarjeta destacada con el préstamo cuya devolución vence primero y accesos rápidos al catálogo y a «Mis préstamos».
RF-02	Catálogo de libros	Listado de libros con título, autor y ejemplares disponibles, filtrable por categoría mediante pestañas o chips.
RF-03	Detalle del libro	Título, autor, año, categoría, sede y ejemplares disponibles, con la acción «Solicitar préstamo» y un diálogo de confirmación.
RF-04	Mis préstamos	Listado de los préstamos del estudiante, ordenado por fecha de devolución más próxima y filtrable por estado (Activo, Devuelto, Vencido).
RF-05	Búsqueda	Campo de búsqueda que filtra el catálogo por título o por autor, sin distinguir mayúsculas ni tildes.
RF-06	Perfil y ajustes	Datos del estudiante y conmutador de tema claro/oscuro que se aplica de inmediato a toda la aplicación.
RF-07	Navegación	Barra de navegación inferior con tres destinos (Inicio, Catálogo, Préstamos), navegación al detalle y retorno correcto con el botón atrás del sistema.

3.1 Reglas de negocio

Las cuatro reglas deben implementarse en la capa de dominio, no en los componentes de la interfaz. Durante la defensa se te pedirá mostrar dónde vive cada una.

Código	Regla
RN-01	Un estudiante no puede tener más de tres préstamos en estado Activo de forma simultánea.
RN-02	No se puede solicitar un libro cuyo número de ejemplares disponibles sea cero.
RN-03	Todo préstamo dura siete días; si la fecha de devolución ya pasó, el préstamo se muestra como Vencido.
RN-04	Un estudiante con al menos un préstamo Vencido no puede solicitar un libro nuevo hasta regularizarlo.

3.2 Estados de interfaz

Toda pantalla que carga datos debe mostrar tres estados distinguibles: carga (con un retardo simulado de 800 milisegundos), contenido y lista vacía. El estado de error se exige únicamente en el catálogo, simulándolo con una bandera que puedas activar desde el repositorio.

3.3 Datos simulados

La fuente de datos en memoria debe contener, como mínimo, la información siguiente. El anexo entrega estos datos listos para copiar.

Conjunto	Contenido mínimo
Estudiante	Un estudiante fijo: nombre, código, carrera y correo.
Categorías	Cinco: Programación, Matemática, Redes, Gestión y Literatura.
Libros	Doce libros repartidos entre las cinco categorías; al menos dos con cero ejemplares disponibles.
Préstamos	Cinco préstamos del estudiante: dos Activos, dos Devueltos y uno Vencido.

4. Requerimientos técnicos obligatorios

Estos requerimientos corresponden a los criterios de evaluación del Producto de la Unidad 1 y se verifican revisando el código fuente.

Ámbito	Exigencia
Entorno multiplataforma	Proyecto KMP con targets Android e iOS configurados y compilando desde el mismo módulo compartido.
Modelado del dominio	Las clases de la sección 4.2 implementadas en commonMain, con el estado del préstamo como sealed class y sin tipos anulables innecesarios.
Interfaz Compose	Pantallas construidas con composables reutilizables, estado elevado (state hoisting) y listas con LazyColumn.
Navegación y theming	Scaffold con barra inferior, rutas tipadas o constantes, tema Material 3 con paleta propia y soporte de modo claro y oscuro.
Arquitectura Clean + MVVM	Capas domain, data y presentation separadas; ViewModel multiplataforma con StateFlow y UiState; casos de uso; interfaz de repositorio en domain e implementación simulada en data.
Inyección de dependencias	Koin configurado, con los módulos declarados en commonMain y la inicialización correspondiente en cada plataforma.
Asincronía	Corrutinas para simular el retardo de carga; ninguna operación bloqueante en el hilo principal.
Diseño responsivo	La interfaz se adapta sin cortes ni superposiciones en teléfono en orientación vertical, como mínimo.

4.1 Estructura de paquetes esperada

```
commonMain/kotlin/pe/upeu/biblioandes/
├── domain/
│   ├── model/           Libro.kt, Prestamo.kt, EstadoPrestamo.kt, Estudiante.kt
│   ├── repository/     BibliotecaRepository.kt (solo la interfaz)
│   └── usecase/         ObtenerCatalogoUseCase.kt, SolicitarPrestamoUseCase.kt,
                        ObtenerPrestamosUseCase.kt
├── data/
│   ├── local/          DatosSimulados.kt (datos en memoria)
│   └── repository/     BibliotecaRepositoryFake.kt
├── presentation/
│   ├── catalogo/       CatalogoViewModel.kt, CatalogoUiState.kt, CatalogoScreen.kt
│   ├── detalle/        DetalleLibroViewModel.kt, DetalleLibroScreen.kt
│   ├── prestamos/      PrestamosViewModel.kt, PrestamosScreen.kt
│   ├── perfil/         PerfilScreen.kt
│   ├── navigation/     AppNavHost.kt, Destinos.kt
│   └── theme/           Color.kt, Type.kt, BiblioAndesTheme.kt
└── di/                 AppModule.kt
```

4.2 Modelo de dominio entregado

Implementa estas clases tal como se presentan. Puedes agregar propiedades, pero no eliminar ni cambiar el tipo de las existentes.

```
// domain/model/Libro.kt
data class Libro(
    val id: Int,
    val titulo: String,
    val autor: String,
    val anio: Int,
    val categoria: String,
    val sede: String,
    val ejemplaresDisponibles: Int
)

// domain/model/EstadoPrestamo.kt
sealed class EstadoPrestamo {
    data class Activo(val diasRestantes: Int) : EstadoPrestamo()
    data class Devuelto(val fechaDevolucion: String) : EstadoPrestamo()
    data class Vencido(val diasDeAtraso: Int) : EstadoPrestamo()
}

// domain/model/Prestamo.kt
data class Prestamo(
    val id: Int,
    val libro: Libro,
    val fechaPrestamo: String,
    val fechaLimite: String,
    val estado: EstadoPrestamo
)
```

Por qué el estado es una sealed class

Cada estado del préstamo lleva información distinta: los días que faltan, la fecha en que se devolvió o los días de atraso. Una cadena de texto o un enum obligarían a arrastrar propiedades vacías. Durante la defensa deberás explicar esta diferencia con tu propio código en pantalla.

5. Trabajo colaborativo con Git

El desarrollo del producto es colaborativo y obligatoriamente trazable. No se acepta un proyecto construido por un solo integrante y subido en un único commit al final: el repositorio debe mostrar quién hizo qué y cuándo. Esta condición se verifica sobre el historial del repositorio y vale dos puntos de la nota final.

Regla no negociable

Cada integrante trabaja en su propia rama y fusiona su aporte mediante una solicitud de incorporación (pull request o merge request) revisada por el compañero. Un integrante sin commits propios de autoría verificable en el repositorio obtiene cero en el examen, aunque la aplicación del equipo funcione perfectamente.

5.1 Modelo de ramas

El repositorio se organiza con el siguiente esquema mínimo. Ninguna funcionalidad se desarrolla directamente sobre main.

Rama	Propósito	Quién escribe en ella
main	Código estable y presentable. Siempre debe compilar y ejecutarse en Android e iOS.	Nadie directamente: solo recibe fusiones aprobadas.
develop	Integración de los aportes del equipo antes de llevarlos a main.	Se actualiza por fusión desde las ramas de funcionalidad.
feature/<funcionalidad>-<apellido>	Desarrollo de una funcionalidad concreta por un integrante.	El integrante responsable de esa funcionalidad.
fix/<descripción>-<apellido>	Corrección de un defecto detectado durante la integración.	Quien detecta o asume la corrección.
sc-<letra>-<apellido>	Solicitud de cambio individual de la Parte II; mínimo tres commits.	El estudiante evaluado, durante la sesión.

Ejemplos válidos de nombres de rama: feature/catalogo-huaman, feature/prestamos-condori, fix/filtro-categoria-huaman, sc-b-condori. Los nombres se escriben en minúsculas, sin tildes ni espacios.

5.2 Flujo de trabajo obligatorio

1. Acuerden en equipo qué funcionalidad asume cada integrante y regístrenlo como issues o tareas en el repositorio.
2. Antes de empezar, actualiza tu copia local: git checkout develop y git pull.
3. Crea tu rama de funcionalidad a partir de develop, nunca a partir de la rama de otro integrante.
4. Trabaja en commits pequeños y frecuentes: al menos un commit por sesión de trabajo, nunca un único commit con todo.
5. Cuando la funcionalidad esté terminada y compile, abre una solicitud de incorporación hacia develop.
6. El otro integrante revisa, comenta y aprueba. Solo entonces se fusiona.

7. Resuelvan los conflictos en la rama de funcionalidad, no en develop.
8. Al cierre de la unidad, fusionen develop en main y etiqueten el commit evaluado como v1.0-unidad1.

```
# Inicio de una funcionalidad
git checkout develop
git pull origin develop
git checkout -b feature/catalogo-huaman

# Trabajo diario
git add .
git commit -m "feat/catalogo): filtrar libros por categoría"
git push -u origin feature/catalogo-huaman

# Integración (tras la revisión del compañero)
git checkout develop
git merge --no-ff feature/catalogo-huaman
git push origin develop

# Cierre de la unidad
git checkout main
git merge --no-ff develop
git tag -a v1.0-unidad1 -m "Producto Unidad 1 - BiblioAndes"
git push origin main --tags
```

5.3 Convención de mensajes de commit

Los mensajes se escriben en español, en presente y describiendo el efecto del cambio, no la acción mecánica. Usa el prefijo que corresponda.

Prefijo	Cuándo se usa	Ejemplo
feat	Se agrega una funcionalidad nueva.	feat(prestamos): filtrar por estado del préstamo
fix	Se corrige un defecto.	fix(detalle): impedir solicitar un libro sin ejemplares
refactor	Se reorganiza el código sin cambiar el comportamiento.	refactor(dominio): extraer el límite de préstamos a un caso de uso
style	Cambios de formato, theming o interfaz sin lógica nueva.	style(tema): ajustar la paleta al modo oscuro
docs	Documentación o README.	docs: describir la estructura de paquetes

Mensajes como «cambios», «avance», «actualización» o «subida final» no cumplen la convención y restan puntaje en el criterio de trabajo colaborativo.

5.4 Evidencia que se revisará

- Gráfico de ramas del repositorio (git log --graph --oneline --all o la vista de red de la plataforma), donde se vea más de una rama de funcionalidad por integrante.
- Listado de contribuciones por autor, obtenido con git shortlog -sne, con aportes de ambos integrantes.

- Al menos dos solicitudes de incorporación cerradas por cada integrante, con al menos un comentario de revisión del compañero.
- La etiqueta v1.0-unidad1 apuntando al commit evaluado.
- Ausencia de commits directos sobre main que no provengan de una fusión.

```
# Comandos que el docente ejecutará sobre el repositorio
git log --graph --oneline --all --decorate | head -40
git shortlog -sne
git branch -a
git log --first-parent main --oneline
```

Cómo se califica el trabajo colaborativo con Git (2 puntos)

2 puntos: ambos integrantes tienen ramas propias, commits distribuidos a lo largo de la unidad, solicitudes de incorporación revisadas y mensajes que cumplen la convención.

1 punto: hay ramas por integrante, pero los commits se concentran en uno o dos días, no hay revisión cruzada o los mensajes son genéricos.

0 puntos: se trabajó directamente sobre main, existe un solo autor en el historial o el repositorio se creó el día de la evaluación.

6. Parte II. Solicitud de cambio del cliente

Durante la evaluación, la dirección de BiblioAndes «solicita» un ajuste. El docente asigna una solicitud distinta a cada integrante del equipo, quien dispone de ciento veinte minutos para implementarla en vivo. Se evalúa que el cambio funcione y que respete la separación de capas: una modificación bien ubicada toca pocos archivos.

Código	Solicitud de cambio	Qué se observa
SC-A	Agregar un chip «Solo disponibles» al catálogo, que oculte los libros sin ejemplares y se combine con el filtro de categoría existente.	Que el filtro se resuelva en el ViewModel o en el caso de uso, no dentro del composible.
SC-B	Mostrar en la barra de navegación inferior un indicador con el número de préstamos activos y deshabilitar el botón «Solicitar préstamo» cuando el estudiante llegue al límite de tres.	Que la regla RN-01 se lea desde el dominio y no se duplique en la interfaz.
SC-C	Permitir ordenar el catálogo por título o por año mediante un selector, conservando el filtro de categoría activo.	Que el ordenamiento sea una operación del dominio y que el estado del selector viva en el UiState.
SC-D	Incorporar la editorial del libro al modelo de dominio, al catálogo y al detalle, mostrándola bajo el nombre del autor.	Que el cambio recorra las tres capas de forma coherente y que los datos semilla se actualicen.

Cómo se califica la Parte II

Funciona y respeta la arquitectura: 4 puntos. Funciona pero con lógica de negocio dentro del composable o duplicada: 2,5 puntos. Cambio incompleto o que rompe otra funcionalidad: 1 punto. No implementado: 0 puntos.

La solicitud de cambio se desarrolla en una rama propia con el nombre `sc-<letra>-<apellido>`, creada a partir de `develop`, y se sube al repositorio antes de finalizar la sesión. Un cambio que no quede versionado no se califica.

Mínimo de commits durante el examen

Cada estudiante debe subir al menos tres commits a su rama durante los ciento veinte minutos del bloque, separados en el tiempo y con mensajes que sigan la convención de la sección 5.3. Un criterio razonable de corte es un commit cuando el cambio compila, otro cuando la funcionalidad opera y un tercero cuando queda limpia y verificada.

Un estudiante que entregue su solicitud de cambio en un único commit final pierde un punto de la Parte II, aunque el cambio funcione correctamente.

7. Parte III. Banco de preguntas de defensa

El docente elige dos preguntas por estudiante. Responder correctamente exige mostrar el código propio, no solo describirlo.

1. Muéstrame dónde vive la regla RN-01 y explica por qué la ubicaste allí y no en la pantalla.
2. Si mañana el servicio web estuviera listo, ¿qué archivos tendrías que crear o modificar para conectarlo? Nómbralos uno por uno.
3. ¿Por qué el estado del préstamo es una `sealed class` y no un `enum` o una cadena de texto?
4. Explica el recorrido completo de un libro desde la fuente simulada hasta el pixel que se dibuja en el catálogo.
5. ¿Qué diferencia hay entre tu `UiState` y tu modelo de dominio? Muéstrame ambos.
6. ¿Dónde se ejecuta la corrutina que simula el retardo de carga y qué ocurre si la pantalla se destruye mientras corre?
7. ¿Por qué el `ViewModel` expone un `StateFlow` y no una variable mutable pública?
8. Muéstrame un composable que hayas hecho reutilizable y explica qué recibe y qué no sabe.
9. ¿Cómo implementaste el cambio de tema claro/oscuro y en qué punto del árbol de composición se aplica?
10. Si la biblioteca agregara una sexta categoría, ¿cuántos archivos tocarías? Justifica la respuesta.
11. Muéstrame tu rama de trabajo y explica qué aportaste tú al producto, qué conflicto tuviste que resolver y cómo lo resolviste.

8. Rúbrica de evaluación

El examen se califica sobre veinte puntos. Los cinco primeros criterios corresponden a los criterios de evaluación del Producto de la Unidad 1 declarados en el sílabo.

Criterio	Pts.	Logrado (100 %)	En proceso (60 %)	Inicial (30 %)
Configuración del entorno multiplataforma	2	El proyecto compila y se ejecuta en Android e iOS desde el mismo módulo compartido.	Compila en ambas plataformas pero requiere ajustes manuales para ejecutarse.	Solo se ejecuta en una plataforma.
Modelado del dominio en commonMain	3	Las clases entregadas implementadas sin alteraciones, estado con sealed class y las cuatro reglas en el dominio.	Clases correctas pero alguna regla de negocio resuelta fuera del dominio.	Modelo alterado o reglas dispersas en la interfaz.
Interfaz Compose funcional	3	Los siete requerimientos funcionales operan, con estados de carga, vacío y error, y composables reutilizables.	Entre cinco y seis requerimientos operan; faltan estados de interfaz.	Menos de cinco requerimientos operan o la interfaz presenta fallas visibles.
Navegación y tema Material 3	2	Navegación completa con retorno correcto, paleta propia y modo claro/oscuro funcional.	Navegación funcional con algún retorno incorrecto, o theming parcial.	Navegación incompleta o tema por defecto sin personalizar.
Organización Clean + MVVM	2	Capas separadas, ViewModel con StateFlow y UiState, casos de uso e interfaz de repositorio en el dominio.	Capas presentes con alguna dependencia invertida o casos de uso ausentes.	La pantalla accede directamente a la fuente de datos.
Trabajo colaborativo con Git	2	Ramas por integrante, commits distribuidos, solicitudes de incorporación revisadas y mensajes según la convención.	Ramas por integrante pero commits concentrados, sin revisión cruzada o con mensajes genéricos.	Trabajo sobre main, un solo autor en el historial o repositorio creado el día de la evaluación.
Solicitud de cambio (Parte II)	4	El cambio funciona, está ubicado en la capa correcta y llegó en al menos tres commits.	El cambio funciona con lógica duplicada, mal ubicada o en un único commit final.	Cambio incompleto o que rompe otra funcionalidad.
Defensa técnica (Parte III)	2	Responde con precisión y sustenta sus decisiones mostrando el código.	Responde parcialmente o describe sin mostrar el código.	No logra explicar el funcionamiento de su propia aplicación.

Nota mínima aprobatoria

Conforme al sílabo, la nota mínima probatoria de la asignatura es trece. Un estudiante que no logre demostrar la ejecución en ambas plataformas no puede superar los catorce puntos, cualquiera sea su desempeño en las demás partes.

9. Entregables

Cada equipo entrega, al término de la sesión:

- Enlace al repositorio Git con la etiqueta «v1.0-unidad1» sobre el commit evaluado.
- Documento PDF con capturas de las cinco pantallas ejecutándose en Android y en iOS.
- Archivo README.md con la estructura de paquetes, las decisiones de arquitectura y las instrucciones de ejecución.
- Rama individual por integrante con la solicitud de cambio de la Parte II («sc-a-apellido», «sc-b-apellido», etc.).
- Captura del gráfico de ramas y del resultado de `git shortlog -sne`, incorporadas al documento PDF de evidencias.
- Enlace a las solicitudes de incorporación cerradas de cada integrante, con los comentarios de revisión visibles.

9.1 Lista de cotejo previa a la evaluación

Verifica estos puntos antes de presentarte. Un «No» en cualquiera de ellos implica perder puntaje.

N.º	Criterio observable	Cumple (Sí / No)
1	La aplicación se ejecuta en Android e iOS desde el proyecto entregado, sin cambios manuales.	
2	Las cinco pantallas del caso están implementadas y son alcanzables por navegación.	
3	Las clases del modelo entregado están implementadas sin cambiar sus tipos.	
4	El estado del préstamo está modelado con una sealed class y no con una cadena de texto.	
5	Las cuatro reglas de negocio están implementadas en la capa de dominio.	
6	Los ViewModel exponen StateFlow y no variables mutables públicas.	
7	Existe una interfaz de repositorio en domain y su implementación simulada en data.	
8	Cada pantalla con datos muestra los estados de carga y vacío, y el catálogo también el de error.	
9	El tema Material 3 usa una paleta propia y responde al modo claro/oscuro.	
10	No hay dependencias de red ni de base de datos en el archivo de compilación.	
11	Cada integrante tiene al menos una rama de funcionalidad propia y commits distribuidos en varias fechas.	
12	Ninguna funcionalidad se subió directamente a main: todo llegó por fusión desde develop.	
13	Cada integrante cerró al menos dos solicitudes de incorporación revisadas por su compañero.	
14	Los mensajes de commit siguen la convención de prefijos y el commit evaluado está etiquetado como v1.0-unidad1.	
15	Durante el examen registré al menos tres commits propios en mi rama, no uno solo al final.	

10. Anexo. Datos semilla entregados

Copia estos datos en tu fuente simulada y complétalos hasta llegar a los doce libros exigidos. Ajusta las fechas para que los préstamos Activos queden siempre en el futuro respecto del día de la evaluación.

```
// data/local/DatosSimulados.kt
object DatosSimulados {

    val estudiante = Estudiante("E-2291", "Diego Huamán Ccama",
        "Ingeniería de Sistemas", "diego.huaman@correo.pe")

    val categorias = listOf("Programación", "Matemática", "Redes",
        "Gestión", "Literatura")

    val libros = listOf(
        Libro(1, "Kotlin en profundidad", "M. Salazar",
            2023, "Programación", "Central", 3),
        Libro(2, "Estructuras de datos", "R. Peña",
            2021, "Programación", "Central", 0),
        Libro(3, "Cálculo aplicado", "L. Ortega",
            2019, "Matemática", "Sede Norte", 2),
        Libro(4, "Redes de computadoras", "A. Medina",
            2022, "Redes", "Sede Sur", 4),
        Libro(5, "Seguridad en redes", "P. Ríos",
            2024, "Redes", "Central", 0),
        Libro(6, "Gestión de proyectos", "S. Delgado",
            2021, "Gestión", "Sede Norte", 2)
        // completa el catálogo hasta doce libros
    )

    val prestamos = listOf(
        Prestamo(1, libros[0], "2026-09-14", "2026-09-21",
            EstadoPrestamo.Activo(5)),
        Prestamo(2, libros[3], "2026-09-15", "2026-09-22",
            EstadoPrestamo.Activo(6)),
        Prestamo(3, libros[2], "2026-08-20", "2026-08-27",
            EstadoPrestamo.Devuelto("2026-08-26")),
        Prestamo(4, libros[1], "2026-08-05", "2026-08-12",
            EstadoPrestamo.Devuelto("2026-08-11")),
        Prestamo(5, libros[5], "2026-08-28", "2026-09-04",
            EstadoPrestamo.Vencido(18))
    )
}
```